Programmation WEB
TP 04 : Javascript (Partie 2)

Consignes :

- ⇨ L'objectif de ces travaux pratiques est de vous apprendre à trouver les informations par vous-mêmes. Le rôle de l'enseignant est de vous épauler dans cette démarche et de vous aiguiller vers les ressources les plus pertinentes.
- ⇨ Ce TP est à réaliser en binôme. Seul un trinôme est autorisé si le nombre d'étudiants est impair. Vous pouvez également travailler seul.
- ⇨ Avant d'entamer chaque TP, des lectures préliminaires/obligatoires sont nécessaires pour s'initier et/ou approfondir les connaissances acquises pendant le cours magistral.
- ⇨ Afin de réaliser les TPs, vous êtes appelés à effectuer des recherches sur Mozilla Developer Network (MDN), Stack Overflow, W3Schools, ou css-tricks (en anglais) peuvent aussi être une bonne ressource.
- ⇨ L'utilisation de ChatGPT est autorisée pour rechercher les erreurs dans votre code si besoin.
- ⇨ Vous avez le choix entre l'utilisation de VSCode, d'un éditeur classique (SublimeText ou Notepad++) et Codepen.

Acquis d'apprentissage :

- ⇨ A la fin de ce TP, vous devez être capable de :
  - Manipuler les objets json et comprendre le format JSON
  - Savoir parcourir un tableau d'objets json
  - Comprendre et utiliser l'historique du navigateur

Sobriété informatique- Green IT

Il s'agit d'implémenter un site web informatif présentant la sobriété informatique - la Green IT et quelques conseils pratiques à adopter dans le développement web ainsi qu'un test de connaissance sur la Green IT.
Ce site comporte 3 pages :

- Accueil.html : cette page permet d'avoir des informations concernant la sobriété informatique (informatique verte – Green IT)
- GreenITSiteWeb.html : cette page communique des informations concernant l'empreinte carbone des sites web
- QuizGreenIT.html : cette page permet de tester les connaissances d'un visiteur sur la Green IT

Point de départ : Code source TP3 ou TP2

Le point de départ de ce TP4 est votre code source du TP2 ou celui du TP3.
N.B : Afin de garder toujours une trace de votre travail, il est fortement recommandé de créer **une copie nommée TP4**.

Objectif :

L'objectif de ce TP est de créer un chatbot interactif en JavaScript, en implémentant trois composantes essentielles : la conception de l'interface utilisateur avec HTML et CSS, la création d'un fichier JSON contenant les intentions du chatbot, et l'écriture du code JavaScript pour gérer l'interaction entre l'utilisateur et le chatbot en récupérant les données du fichier JSON et affichant les réponses dans une boîte de chat.

Etape 1 : Mise en place de l'interface du chatbot

Reprenons la page GreenITSiteWeb.html.

1- Ajoutez une nouvelle section avant celle contenant le tableau.
2- Mettez en place l'interface ci-dessous en y ajoutant les éléments nécessaires : un div vide pour la boite de chat, un input pour le champ de saisie et enfin le bouton d'envoi.
3- Utilisez le CSS pour styliser l'apparence de l'interface utilisateur. Vous pouvez définir les couleurs, les polices, les marges, etc., pour rendre votre chatbot attrayant et convivial.
    a. Donnez l'id 'chat-box' au div qui contiendra le chat l'id 'user-input' à la balise input.

Etape 2 : Création du fichier JSON et du script

Lectures préalables :

- CM : Javascript
- MDN : Les objets JS
- MDN : format JSON
- MDN : l'api fetch
- MDN : les « promises »

Le fichier JSON

1- Créez un sous-dossier json au même niveau que les sous-dossier html, css et js
2- Accédez au dossier créé et créez un fichier JSON nommé **intents.json**
3- Le fichier intents.json contiendra les intentions du chatbot sous forme d'objets JSON avec des propriétés telles que tag (mot clé), patterns (motifs) et responses (réponses). Vous pouvez récupérez le fichier disponible sur Moodle ou créer vous-même votre fichier json. Assurez-vous que le fichier JSON est correctement structuré et valide.

Le script

1- Dans le dossier js, créez un nouveau fichier chatbot.js et reliez-le au fichier GreenITSiteWeb.html
2- Au niveau du bouton « Envoyer », ajouter à l'événement onclick l'action fetchJSON('../json/intents.json'). Cette fonction prenant en paramètre l'url du fichier json sera implémentée dans la suite. Elle permet de récupérer le contenu du fichier json et de traiter les données stockées.
3- Dans le fichier chatbot.js, ajoutez la fonction fetchJSON(url) ci-dessous :

```
// Fonction pour récupérer et traiter le JSON
function fetchJSON(url) {
    // Récupérer le JSON à partir de l'URL fournie
    fetch(url)
    //then est une méthode qui retourne une promesse et prend en paramètre une
       //fonction callback qui sera exécutée une fois la promesse résolue
      .then(response => {
        // Vérifier si la réponse est correcte
        if (!response.ok) {
            // Si la réponse n'est pas correcte, lancer une erreur
          throw new Error('Network response was not ok');
        }
        // Si la réponse est correcte, retourner le JSON
        return response.json();
      })
     //then ici permettra de récupérer le JSON retourné par la promesse
      .then(data => {
        // Vérifier si le JSON est vide ou mal formé
        if (Object.keys(data).length === 0 && data.constructor === Object) {
            // Si le JSON est vide ou mal formé, lancer une erreur
            throw new Error('Empty JSON or malformed JSON');
          }
```

```
      //On affiche le JSON dans la console. Il s'agit d'un objet contenant les
      // intentions du chatbot
       console.log(data);

     // Passer les intentions à la fonction sendMessage qui sera définie plus tard
       sendMessage(data.intents);
     })
      //catch est une méthode qui retourne une promesse et prend en paramètre une
      //fonction callback qui sera exécutée en cas d'erreur
     .catch(error => {
       // En cas d'erreur, afficher un message d'erreur dans la console
       console.error('There was a problem with the fetch operation:', error);
     }) ;
}
```

4- Implémentez la fonction sendMessage (Intents). Elle prend en paramètre les intentions du chatbot récupérés à partir du fichier json. Voici l'algorithme qu'on souhaite mettre en place :
   a. Récupération de la saisie de l'utilisateur dans une variable.
   b. Affichage du message de l'utilisateur dans la boîte de chat
   c. Traitement du message de l'utilisateur et obtention la réponse du chatbot
   d. Affichage de la réponse du chatbot dans la boîte de chat
   e. Enfin, il faut effacer le champ de saisie

Astuce : Mettez en place les deux fonctions suivantes

- *showMessage(message, type) :* message sera le texte à ajouter dans le chatbot et type représentera une chaine de caractère qui peut être soit 'user', soit 'bot'. Cette fonction sera utilisée aux étapes b. et d. de la fonction sendMessage.

- *processMessage(intents, messageUser)* qui prend en paramètres les intentions du chatbot ainsi que l'entrée utilisateur récupérée à l'étape a. et renvoie la réponse du bot. Voici le code de cette fonction :

```
// Fonction pour traiter le message de l'utilisateur
function processMessage(intents, message) {
    // Par défaut, la réponse est "Je suis désolé, je ne suis pas sûr de
comprendre."
    let response = "Je suis désolé, je ne suis pas sûr de comprendre.";
    // Parcourir les intentions du chatbot
    intents.forEach(intent => {
      // Vérifier si le message de l'utilisateur correspond à l'un des motifs
      intent.patterns.forEach(pattern => {
        // Vérifier si le message de l'utilisateur contient le motif
        if (message.toLowerCase().includes(pattern.toLowerCase())) {
```

```
            // Sélectionner une réponse aléatoire parmi les réponses possibles
            response = intent.responses[Math.floor(Math.random() *
intent.responses.length)];
        }
      });
    });
    // Retourner la réponse
   return response;
}
```

Voici un exemple de fonctionnement du chat :

Exemple ChatBot -GreenIT

Etape 3 : Récupérer l'historique de la session précédente du chat

Lectures préalables :

- MDN : Session Storage
- MDN : EventTarget
- MDN : EventListener
- MDN : querySelector

L'objectif ici est d'ajouter la fonctionnalité de sauvegarde de l'historique des messages à l'aide de *sessionStorage*.

1- Au début du fichier chatbot.js, déclarez une classe nommée *ChatHistory* avec un constructeur vide.
2- Dans le constructeur, initialisez une propriété messages à un tableau vide pour stocker les messages.
3- Implémentez une méthode nommée *addMessage(message)* qui prend un paramètre message et l'ajoute à la fin du tableau messages.
4- Créez une méthode nommée *getHistory()* qui renvoie le tableau messages contenant l'historique complet des messages.
5- En utilisant addEventListener, enregistrez les messages du chat dans la session du navigateur avanyt de quitter la page
6- Puis en utilisant toujours addEventListener, chargez les messages de la session du navigateur au chargement de la page.

Voici les deux fonctions à utiliser :

```
// création d'une session navigateur pour stocker les messages
function saveMessages() {
        console.log('Saving chat history...');
        console.log(historyMessages.getHistory());
        sessionStorage.setItem('chatHistory',
        JSON.stringify(historyMessages.getHistory().map(msg => msg.message)));
}
```

```
// Fonction pour charger les messages de la session navigateur
function loadMessages() {
	// Récupérer l'historique des messages de la session navigateur
	const chatHistory = JSON.parse(sessionStorage.getItem('chatHistory'));
	if (chatHistory) {
		chatHistory.forEach(message => {
			showMessage(message, message.sender);
		});
	}
}
```

Et voici donc le fonctionnement de l'historique

SessionStorage